

Moodle Web Services

curl Test Report

Date: 2026-09-28 | Moodle 5.0.1 | http://localhost:9999 | Protocol: REST / JSON

End-to-end test of the full read -> download -> grade -> verify path using plain curl.
Shell variables used: TOKEN=da3802b9af3f9893e106c9579e2a8132 | URL=http://localhost:9999/webservice/rest/server.php
Every POST sends: wstoken, wsfunction, moodlewsrestformat=json. Errors return HTTP 200 with an 'exception' JSON body.

Web Service Functions Used

Function / Endpoint	What it does
core_webservice_get_site_info	Token health check - verify token + list callable functions
mod_assign_get_assignments	List assignments in a course (id, cmid, max grade, files)
mod_assign_get_submissions	List all student submissions with file download URLs
mod_assign_get_submission_status	Detailed per-student view including feedback after grading
mod_assign_get_grades	Read stored numeric grades for an assignment
mod_assign_save_grade	Write grade + feedback to assign table AND gradebook
mod_assign_list_participants	List enrolled students with submission/grading state
core_files_get_files	Browse a file area directly by contextid/component
pluginfile.php?token= (GET)	Plain HTTP file download - not a wsfunction

Test Steps

Step 1 - core_webservice_get_site_info

PASS

PURPOSE

Verify the token is valid and list which functions it can call.

CURL COMMAND

```
curl -s "$URL" \  
  -d "wstoken=$TOKEN" \  
  -d "wsfunction=core_webservice_get_site_info" \  
  -d "moodlewsrestformat=json"
```

RESPONSE

```
{  
  "username": "admin",  
  "userid": 2,  
  "functions": [  
    {"name": "core_webservice_get_site_info"},  
    {"name": "mod_assign_get_assignments"},  
    {"name": "mod_assign_get_submissions"},  
    {"name": "mod_assign_get_submission_status"},  
    {"name": "mod_assign_get_grades"},  
    {"name": "mod_assign_save_grade"},  
  ]  
}
```

```
{
  "name": "mod_assign_list_participants",
  "name": "core_files_get_files"
},
"downloadfiles": 1,
"release": "5.0.1 (Build: 20250609)"
}
```

NOTES

Token valid. Belongs to admin (userid 2). All 8 functions registered. downloadfiles=1 confirms file download is enabled.

Step 2 - mod_assign_get_assignments

PASS

PURPOSE

Find assignment id, cmid, max grade, due date, and teacher-attached files.

CURL COMMAND

```
curl -s "$URL" \  
-d "wstoken=$TOKEN" \  
-d "wsfunction=mod_assign_get_assignments" \  
-d "moodlewsrestformat=json" \  
-d "courseids[0]=2"
```

RESPONSE

```
{  
  "courses": [{  
    "id": 2, "fullname": "python",  
    "assignments": [{  
      "id": 2, "cmid": 3, "name": "basics",  
      "grade": 100, "markingworkflow": 0,  
      "introattachments": [{  
        "filename": "Python_Basics_Assignment.ipynb",  
        "filesize": 27244  
      }]  
    }]  
  }]  
}
```

NOTES

assignmentid=2 (used in all remaining calls). cmid=3. Max grade 100. Feedback comments enabled. markingworkflow=0. Question notebook attached (27,244 bytes).

Step 3 - mod_assign_get_submissions

PASS

PURPOSE

List all submissions - who submitted, their status, and the file download URLs.

CURL COMMAND

```
curl -s "$URL" \  
-d "wstoken=$TOKEN" \  
-d "wsfunction=mod_assign_get_submissions" \  
-d "moodlewsrestformat=json" \  
-d "assignmentids[0]=2"
```

RESPONSE

```
{  
  "assignments": [{  
    "assignmentid": 2,  
    "submissions": [  
      {"id": 3, "userid": 2, "status": "new", "files": []},  
      {"id": 4, "userid": 3, "status": "submitted",  
        "files": [{  
          "filename": "Python_Basics_Assignment_ANSWER_KEY.ipynb",  
          "filesize": 27244  
        }]  
      }  
    ]  
  }]  
}
```

```
        "filesize":42552,  
        "fileurl":"http://localhost:9999/webservice/  
        pluginfile.php/21/assignsubmission_file/  
        submission_files/4/...ipynb"  
    }  
}  
]  
}]  
}
```

NOTES

Sai (userid=3) submitted Python_Basics_Assignment_ANSWER_KEY.ipynb (42,552 bytes). submission id=4, attemptnumber=0.
Admin (userid=2) has no files (status: new).

Step 4 - mod_assign_get_submission_status (BEFORE grading)

PASS

PURPOSE

Per-student baseline snapshot - confirms graded=false before writing anything.

CURL COMMAND

```
curl -s "$URL" \
  -d "wstoken=$TOKEN" \
  -d "wsfunction=mod_assign_get_submission_status" \
  -d "moodlewsrestformat=json" \
  -d "assignid=2" \
  -d "userid=3"
```

RESPONSE

```
{
  "gradingsummary": {
    "submissionsneedgradingcount": 1
  },
  "lastattempt": {
    "graded": false,
    "gradingstatus": "notgraded"
  }
}
```

NOTES

graded=false / gradingstatus=notgraded / submissionsneedgradingcount=1. No feedback block present. Compare to Step 8.

Step 5 - pluginfile.php (GET - file download)

PASS

PURPOSE

Download the student's submitted .ipynb byte-for-byte. Not a wsfunction - plain GET with token in query string.

CURL COMMAND

```
curl -s -o /tmp/sai_submission.ipynb \
  "http://localhost:9999/webservice/pluginfile.php/21/
assignsubmission_file/submission_files/4/
Python_Basics_Assignment_ANSWER_KEY.ipynb
?token=da3802b9af3f9893e106c9579e2a8132" \
  -w "HTTP %{http_code}, size %{size_download} bytes"
```

RESPONSE

```
HTTP 200, size 42552 bytes
```

NOTES

Notebook downloaded intact. Moodle serves .ipynb as application/json. URL pattern: .../pluginfile.php/<contextid>/assignsubmission_file/submission_files/<submissionid>/<filename>?token=...

Step 6 - mod_assign_save_grade

PASS

PURPOSE

Write grade (85/100) + feedback comment. Writes to both mdl_assign_grades and the course gradebook in one call.

CURL COMMAND

```
curl -s "$URL" \
  -d "wstoken=$TOKEN" \
  -d "wsfunction=mod_assign_save_grade" \
  -d "moodlewsrestformat=json" \
  -d "assignmentid=2" -d "userid=3" \
  -d "grade=85" -d "attemptnumber=0" \
  -d "addattempt=0" -d "workflowstate=" \
  -d "applytoall=0" \
  --data-urlencode \
  "plugindata[assignfeedbackcomments_editor][text]=
  <p>Good work on Python basics. Score: 85/100.</p>" \
  -d "plugindata[assignfeedbackcomments_editor][format]=1"
```

RESPONSE

```
null
```

NOTES

null = success. Failures return a JSON exception object. grade is on the assignment's own scale (max 100). workflowstate empty = no workflow. Use readyforreview when marking workflow is on.

Step 7 - mod_assign_get_grades

PASS

PURPOSE

Read the stored grade back - confirms the write in Step 6 landed in the assignment table.

CURL COMMAND

```
curl -s "$URL" \
  -d "wstoken=$TOKEN" \
  -d "wsfunction=mod_assign_get_grades" \
  -d "moodlewsrestformat=json" \
  -d "assignmentids[0]=2"
```

RESPONSE

```
{
  "assignments": [{
    "assignmentid": 2,
    "grades": [{
      "id": 1, "userid": 3,
      "attemptnumber": 0,
      "grader": 2,
      "grade": "85.00000"
    }]
  }]
}
```

NOTES

grade=85.00000 for userid=3 / attempt 0. grader=2 (admin) is the audit trail - in production this field shows the dedicated LLM Grader user.

Step 8 - mod_assign_get_submission_status (AFTER grading)

PASS

PURPOSE

Confirm the student-facing view: grade display, feedback comment, graded=true.

CURL COMMAND

```
curl -s "$URL" \  
  -d "wstoken=$TOKEN" \  
  -d "wsfunction=mod_assign_get_submission_status" \  
  -d "moodlewsrestformat=json" \  
  -d "assignid=2" \  
  -d "userid=3"
```

RESPONSE

```
{  
  "gradingsummary": {  
    "submissionsneedgradingcount": 0  
  },  
  "lastattempt": {  
    "graded": true,  
    "gradingstatus": "graded"  
  },  
  "feedback": {  
    "grade": {"grade": "85.00000"},  
    "gradeformatdisplay": "85.00 / 100.00",  
    "plugins": [{"type": "comments",  
      "editorfields": [{"text":  
        "<p>Good work on Python basics.  
        Score: 85/100.</p>"}]}  
    }  
  }  
}
```

NOTES

graded=true / gradingstatus=graded / submissionsneedgradingcount=0. Student sees 85.00 / 100.00 and the feedback comment.

Before vs After Comparison (Step 4 -> Step 8)

Field	Step 4 - Before grading	Step 8 - After grading
graded	false	true
gradingstatus	notgraded	graded
submissionsneedgradingcount	1	0
feedback block	absent	grade + comment present
gradeordisplay	-	85.00 / 100.00

Final Summary

#	Function	Result	Status
1	core_webservice_get_site_info	Token valid (admin, userid 2), 8 functions available	PASS
2	mod_assign_get_assignments	assignmentid=2 / cmid=3 / max 100 / question notebook present	PASS
3	mod_assign_get_submissions	Sai (userid 3) submitted answer key .ipynb (42,552 bytes)	PASS
4	mod_assign_get_submission_status	Baseline: graded=false, gradingstatus=notgraded	PASS
5	pluginfile.php?token= (GET)	Notebook downloaded, HTTP 200, 42,552 bytes	PASS
6	mod_assign_save_grade	Returned null (success) - grade 85 + comment written	PASS
7	mod_assign_get_grades	grade=85.00000 confirmed in assignment table	PASS
8	mod_assign_get_submission_status	Student sees 85.00/100.00 + feedback comment	PASS

Production Notes

Token security	Do NOT use the admin token in production. Create a dedicated 'LLM Grader' user with a Teacher role scoped to the relevant courses, its own token, an IP restriction, and an expiry date.
Marking workflow	Set workflowstate=readyforreview when the assignment has marking workflow enabled. This hides the LLM grade from the student until a teacher reviews and releases it.
attemptnumber	Always send the real attempt number (e.g. 0), not -1, in automated code. -1 means 'latest' but makes staleness checks harder.
Audit trail	The grader field in mod_assign_get_grades shows which user wrote each grade. Use a dedicated LLM user so human vs LLM grades are distinguishable.
Token persistence	Web services and the token must be re-enabled after a container restart. Enable them permanently via the Moodle admin UI or a provisioning script.
File URL pattern	.../pluginfile.php/<contextid>/assignsubmission_file/submission_files/<submissionid>/<filename>?token=. ..
Error detection	All WS calls return HTTP 200 even on error. Always check the response body for an 'exception' key - do not rely on HTTP status alone.